

Viabilidade de uma Transição Pós-Quântica Híbrida na Blockchain Solana via Syscall Nativo

Arthur Tsukamoto Oliveira¹, Bryan Kano Ferreira¹, Ana Cristina dos Santos¹

Instituto de Tecnologia e Liderança (Inteli) São Paulo - SP - Brasil

arthur.oliveira@sou.inteli.edu.br bryan.ferreira@prof.inteli.edu.br anacris.santos@prof.inteli.edu.br

Abstract—Computadores quânticos ameaçam assinaturas como o Ed25519, que sustenta a Solana. Investigamos uma coexistência híbrida em vez de migração completa, combinando chaves Falcon-512 *off-chain*, vinculação *on-chain* via PDA e um *syscall* nativo de verificação no *runtime* do validador. Em 30.540 transações, a verificação direta na SVM mostrou-se impraticável, enquanto a via *syscall* é viável, e o principal *trade-off* passa a ser o tamanho da transação e a integração *on-chain*. Sob arquitetura equivalente, o Ed25519 verificado pelo mesmo *syscall* atinge paridade ($1.03\times$) com o Falcon-512, com diferença de 238 *Compute Units*, indicando custo de origem arquitetural, não da primitiva pós-quântica.

Keywords—Criptografia pós-quântica, Blockchain, Solana, Falcon-512, Criptografia híbrida

Abstract—Quantum computers threaten signatures such as Ed25519, which underpins Solana. We investigate hybrid coexistence rather than full migration, combining off-chain Falcon-512 keys, on-chain binding via a PDA, and a native verification *syscall* in the validator runtime. Across 30,540 transactions, direct verification inside the SVM is impractical, while the *syscall* path is viable, and the main trade-off shifts to transaction size and on-chain integration. Under an equivalent architecture, Ed25519 verified by the same *syscall* reaches parity ($1.03\times$) with Falcon-512, differing by 238 Compute Units, indicating that the cost is architectural rather than from the post-quantum primitive.

Keywords—Post-quantum cryptography, Blockchain, Solana, Falcon-512, Hybrid Cryptography

I. INTRODUÇÃO

O algoritmo de Shor estabelece que os problemas da fatoração de inteiros e do logaritmo discreto podem ser resolvidos em tempo polinomial por um computador quântico relevante [1], tornando vulneráveis os esquemas criptográficos de chave pública que dependem desses problemas como fator de segurança. Esse cenário possibilita o risco de estratégias adversariais do tipo *Harvest Now, Decrypt Later*, nas quais dados capturados hoje podem ser explorados quando a capacidade quântica estiver disponível [2]. Em *blockchains*, a principal vulnerabilidade se manifesta na exposição das chaves públicas dos usuários na cadeia, possibilitando que um adversário com um computador quântico criptograficamente relevante possa derivar a chave privada correspondente [3].

A Solana depende do esquema de assinatura digital Ed25519 para autenticação de transações e para componentes centrais de sua arquitetura de execução [4], [5]. Essa escolha é coerente com uma rede de alta vazão, pois combina assinaturas compactas com baixo custo computacional. Entretanto, como

o Ed25519 se baseia no problema do logaritmo discreto em curvas elípticas, sua segurança deixa de ser suficiente diante de computadores quânticos capazes de executar o algoritmo de Shor [1]. A migração para criptografia pós-quântica, contudo, envolve desafios adicionais. Além de preservar compatibilidade operacional, a Solana impõe um limite de 1.232 *bytes* por transação, o que penaliza esquemas com chaves e assinaturas grandes [6]. Em paralelo, a publicação dos primeiros padrões FIPS de criptografia pós-quântica pelo NIST reforça a relevância de estratégias de adoção gradual e compatível com sistemas legados [8].

Nesse contexto, o Falcon-512 se destaca por combinar assinatura relativamente compacta, cerca de 666 bytes, com segurança pós-quântica baseada em reticulados [9], tendo sido selecionado pelo NIST para padronização como FND-DSA. Ainda assim, a adoção direta desse algoritmo esbarra em uma restrição fundamental da *Solana Virtual Machine*. Programas *on-chain* são executados em um ambiente derivado de eBPF/SBF, no qual operações de ponto flutuante têm suporte apenas limitado e emulado por *software*, não nativo [10]. Como a implementação do Falcon-512 depende desse tipo de aritmética [16], a verificação direta dentro da SVM mostra baixa aderência às restrições atuais do ambiente.

Este trabalho investiga, portanto, não uma transição completa da rede Solana, mas uma abordagem de coexistência criptográfica híbrida. A proposta combina Ed25519 para compatibilidade nativa com Falcon-512 como camada adicional de autenticação, vinculando a chave pública pós-quântica a uma PDA e deslocando a verificação criptográfica para o *runtime* do validador por meio de um *syscall* nativo. Diante disso, as contribuições do trabalho podem ser resumidas em três frentes. A primeira examina experimentalmente as limitações da verificação Falcon-512 dentro da *Solana Virtual Machine* (SVM). A segunda descreve a integração de um *syscall* nativo para essa verificação. Por fim, a terceira caracteriza os efeitos da abordagem sobre o tempo de assinatura, o tempo de confirmação, o tamanho de transação e o consumo de *Compute Units*.

II. TRABALHOS RELACIONADOS

Do ponto de vista da literatura, a integração de criptografia pós-quântica em *blockchains* já foi discutida sob diferentes estratégias. Yang et al. [11] mostram que o desafio dominante é conciliar segurança pós-quântica com custos de computação

e comunicação. Na Ethereum, a LACChain reporta verificação *on-chain* de assinaturas Falcon-512, mas com alto custo quando toda a lógica permanece na camada de contrato [12]. No mesmo ecossistema do Ethereum, o EIP-7619 propõe um precompilado para verificação genérica de assinaturas Falcon-512, enquanto o EIP-8052 propõe suporte a Falcon por meio de um precompilado nativo, ambos deslocando a operação criptográfica para fora da máquina virtual e reduzindo a dependência de contratos para a etapa de verificação [13], [14]. Já a Algorand relata transações pós-quânticas com Falcon-1024, porém com parâmetros que não se acomodam ao limite de tamanho de transação da Solana [15].

No ecossistema da própria Solana, duas propostas recentes abordam diretamente o problema. A Anza propõe a adição de Falcon-512 via precompilado SIMD-0461, deslocando a verificação para fora da SVM, mas sem avaliação experimental publicada [17]. O Jump Crypto recomenda o uso de *syscalls* nativos como mecanismo preferencial para integração pós-quântica no *runtime* do validador, contudo, sem apresentar *benchmark* concreto [18].

Em comparação com essas abordagens, a contribuição deste trabalho está menos na proposta de um novo esquema criptográfico e mais na adaptação do mecanismo de verificação ao contexto da Solana, conforme sintetizado na Tabela I. O foco está em um cenário híbrido, no qual a camada nativa da rede continua baseada em Ed25519, enquanto Falcon-512 é introduzido como mecanismo adicional de autenticação compatível com as restrições de tamanho e execução observadas no *runtime*, sem demonstrar a substituição integral da lógica criptográfica da rede.

TABELA I

COMPARAÇÃO RESUMIDA COM TRABALHOS RELACIONADOS.

Abordagem	Integração	Execução	Limitação principal
Yang et al. [11]	<i>Survey</i> de <i>blockchains</i> PQC	–	Identifica desafios, mas não avalia Solana.
LACChain [12]	Falcon-512 em Besu	Contrato	Alto custo quando a verificação fica na camada de contrato.
EIP-7619 / EIP-8052 [13], [14]	Precomp.	Fora da EVM	Propostas voltadas ao ecossistema Ethereum.
Algorand [15]	Falcon-1024	Mecanismo nativo	Parâmetros incompatíveis com o limite de 1.232 bytes da Solana.
Anza [17]	Precomp. SIMD-0461	Fora da SVM	Proposta arquitetural, sem avaliação experimental publicada.
Jump Crypto [18]	<i>Syscall</i> Falcon-512	<i>Runtime</i>	Recomenda <i>syscalls</i> , sem <i>benchmark</i> publicado.
Este trabalho	PDA + <i>syscall</i> nativo	<i>Runtime</i> do validador	Avaliação restrita a ambiente <i>single-node</i> .

III. ESCOLHA DO ALGORITMO

A SVM executa programas em um ambiente restrito, com limite de *Compute Units* por instrução e por transação, além das limitações herdadas do modelo eBPF/SBF [10]. Na Solana,

essas restrições interagem diretamente com o problema pós-quântico. Se a chave pública Falcon-512 e a assinatura forem transmitidas juntas, o total supera o limite de 1.232 bytes por transação. No desenho avaliado neste trabalho, a chave pública é armazenada em um *Program Derived Address* (PDA), o que reduz o *payload* transmitido para a assinatura, o *hash* da mensagem e os metadados da instrução [6], [7]. Esse ajuste faz com que o Falcon-512 seja o único candidato considerado neste recorte que permanece compatível com o limite da rede quando a chave pública não viaja em toda transação.

TABELA II

COMPATIBILIDADE DE ALGORITMOS COM O LIMITE DE 1.232 BYTES POR TRANSAÇÃO.

Algoritmo	PK	Sig	Compatível?
Ed25519	32	64	Sim
Falcon-512	897	666	Não
Falcon-512 (PK em PDA)	–	666	Sim
Falcon-1024	1.793	1.280	Não
ML-DSA-44	1.312	2.420	Não
SLH-DSA-128f	32	17.088	Não

Nessa análise, resumida na Tabela II, o Falcon-512 é o único esquema pós-quântico compatível com a Solana ao combinar a assinatura de 666 bytes com armazenamento persistente da chave pública em PDA. Mesmo com essa otimização, a escolha envolve um *trade-off* estrutural, pois a verificação Falcon-512 depende da amostragem gaussiana discreta sobre reticulados NTRU, que requer aritmética de ponto flutuante de dupla precisão (IEEE 754) por estabilidade numérica [16]. Trata-se de característica intrínseca ao algoritmo, e não de limitação introduzida pelo ambiente eBPF/SBF da SVM [10], o que reforça a inadequação de embutir a verificação diretamente em programas *on-chain* sem suporte de mecanismos nativos do *runtime*.

IV. METODOLOGIA

O estudo foi conduzido em duas etapas complementares. Na primeira, foi implementada uma arquitetura híbrida com geração *off-chain* do par de chaves Falcon-512, registro da chave pública em uma PDA derivada da identidade Ed25519 do usuário e submissão de transações com dupla autenticação. Essa etapa buscou mostrar que o armazenamento da chave pública pós-quântica em estado *on-chain* é viável e verificar experimentalmente a limitação da SVM para executar a verificação Falcon-512 diretamente no programa.

O particionamento entre componentes *on-chain* e *off-chain* decorre do limite de 1.232 bytes por transação, que impede transmitir a chave pública Falcon-512 junto com a assinatura. O cliente *off-chain* gera as chaves, assina localmente e constrói as transações com apenas o *hash*, a assinatura e os metadados necessários, enquanto o programa *on-chain* recupera a chave pública da PDA e aciona a verificação nativa.

A conta PDA possui um *layout* determinístico de 955 bytes: 8 bytes de discriminador, 32 bytes para a chave pública Ed25519 do proprietário, 897 bytes para a chave pública Falcon-512, 8 bytes para um *timestamp* de criação (*i64*), 8 bytes para um *nonce* monotônico (*u64*), 1 byte de *flag* de inicialização e 1 byte para o *canonical bump*

persistido (usado na derivação determinística da PDA via `create_program_address`). Esse *layout* vincula criptograficamente a identidade Ed25519 do usuário à chave pública pós-quântica armazenada, sem permitir reescrita por contas não autorizadas pelo programa [7].

No nível lógico, o programa híbrido organiza o fluxo em três operações. A primeira registra a chave pública Falcon-512 do usuário e estabelece o vínculo entre sua identidade Ed25519 e a PDA controlada pelo programa. A segunda permite verificar uma assinatura pós-quântica sem transferência de fundos, isolando a etapa criptográfica da lógica de negócio. A terceira combina verificação e movimentação de *lamports*, permitindo que a autorização pós-quântica participe do fluxo de transferência. Nessa operação, a assinatura Falcon-512 cobre um *digest* que vincula o destinatário, o valor, a PDA e um *nonce* monotônico mantido na conta. O programa recomputa esse *digest on-chain* a partir dos parâmetros reais da transação e do *nonce* corrente, rejeitando tanto a substituição de parâmetros quanto o reúso de assinaturas (*replay*).

Na segunda etapa, a verificação criptográfica foi deslocada para o *runtime* do validador por meio de um *syscall* nativo, `sol_falcon512_verify`, implementado em um *fork* do Agave (Principal Repositório da Solana).¹ O *syscall* recebe ponteiros e comprimentos da mensagem, da assinatura e da chave pública, realiza o mapeamento da memória da SVM, valida estruturalmente os componentes criptográficos e delega a verificação à biblioteca `pqcrypto-falcon`. Códigos de retorno distintos são utilizados para sucesso, chave pública inválida, assinatura malformada e falha de verificação, permitindo desacoplar erro estrutural de erro criptográfico e aplicar políticas diferentes para cada caso no programa *on-chain*.

O custo do *syscall* foi calibrado com 10.000 medições, seguindo a mesma lógica de conversão usada para *syscalls* nativas da Solana, com referência de cerca de 33 ns por *Compute Unit*. A Tabela III resume a distribuição da verificação Falcon-512. Adotou-se 900 CU, equivalente ao P99 acrescido de margem de 30%, como custo conservador do *syscall*.

TABELA III

CALIBRAÇÃO DO CUSTO DO *syscall* `SOL_FALCON512_VERIFY` (1 CU $\approx$ 33 ns).

Métrica	Tempo (ns)	CU
Média	17.123	519
Mediana	17.042	516
Desvio padrão	1.537	47
IC 95% (média)	[17.093; 17.153]	[518; 520]
P95	18.708	567
P99	21.917	664
P99 + 30%	28.492	863
Valor adotado	–	900

O *benchmark* comparou três abordagens implementadas para transferências de 1.000 *lamports* em um `solana-test-validator` modificado. A primeira corresponde à transferência nativa autenticada por Ed25519. A segunda corresponde à transferência autorizada por assinatura

Falcon-512 verificada pelo *syscall* nativo. A terceira corresponde ao Ed25519 sob a mesma arquitetura do Falcon-512, verificado por um *syscall* nativo dedicado. Ao todo, foram avaliadas 30.540 transações, distribuídas em lotes de $N = 5, 25, 50, 100$ e 10.000, medindo tempo de assinatura, tempo de confirmação e *Compute Units*. Como a verificação Ed25519 nativa é contabilizada fora do *meter* do programa, a análise distingue entre custo reportado e custo criptográfico efetivo para refletir essa disparidade de medição.

Para isolar o custo da primitiva criptográfica do custo da integração arquitetural, foi implementado um terceiro cenário, o Ed25519 sob a mesma arquitetura do Falcon-512 (PDA armazenando a chave pública, verificação via *syscall* nativo e transferência direta de *lamports*). Para isso, adicionou-se ao *runtime* do validador um *syscall* `sol_ed25519_verify` simétrico ao do Falcon-512, com custo calibrado pelos mesmos critérios (`SIGNATURE_COST` oficial de 720 *Compute Units*). Em ambos os fluxos, a derivação da PDA é determinística, com o *canonical bump* persistido na conta e a verificação feita por `create_program_address`, o que garante custo constante. Essa implementação permite medir empiricamente, e não apenas estimar, a diferença entre os dois fluxos sob arquitetura idêntica.

V. RESULTADOS

Os experimentos confirmaram uma taxa de sucesso de 100% em todas as execuções, com métricas estáveis em todos os lotes ($N = 5, 25, 50, 100$ e 10.000), confirmando que o custo adicional do fluxo Falcon-512 é inerente ao mecanismo, e não à carga experimental. Os valores de referência adotados para a comparação entre os três cenários são os do maior lote, $N = 10.000$, sintetizados na Figura 1 e detalhados na Tabela V.

O tempo de assinatura do Falcon-512 (mediana de 143 μ s) foi cerca de 6 vezes maior que o do Ed25519, mas continuou na ordem de microssegundos e representou fração pequena da latência total observada. O tamanho da transação subiu de 215 para 902 bytes, permanecendo dentro do limite de 1.232 bytes. Os tempos de confirmação foram estatisticamente equivalentes (mediana de 502 ms para os três cenários, com a diferença entre Ed25519 sob mesma arquitetura e Falcon-512 não significativa) para $N = 10.000$.

A decomposição do consumo de *Compute Units* do fluxo Falcon-512 mostra que, dos 7.900 CU totais, apenas 900 CU (11,4%) correspondem à verificação criptográfica realizada pelo *syscall*. O restante distribui-se entre o *overhead* do *runtime* BPF, a leitura e a derivação da PDA, chamadas de *log* e demais operações arquiteturais. Para garantir medições reproduzíveis, a derivação da PDA é determinística, com o *canonical bump* persistido na conta durante o registro e a verificação feita por `create_program_address`, o que mantém o custo total constante (CV de 0,02%) e permite uma comparação direta entre os dois fluxos. Os *logs* dos dois fluxos são idênticos, de modo que a comparação reflete apenas o custo do mecanismo.

A Tabela IV detalha esse consumo por componente, medido com o *syscall* `sol_remaining_compute_units` em pontos de checkpoint do programa, e o contrasta com o

¹Implementação, dados coletados e scripts disponíveis em: https://anonymous.4open.science/r/agave_falcon512-3C25

cenário Ed25519 sob arquitetura equivalente. Os componentes arquiteturais (*runtime* BPF, leitura e derivação da PDA, *logs* e transferência) são praticamente idênticos nos dois fluxos. Em termos absolutos, o Ed25519 sob a mesma arquitetura consumiu 7.662 CU contra 7.900 CU do Falcon-512, uma paridade técnica de $1,03\times$. A diferença total de apenas 238 CU concentra-se sobretudo na primitiva de verificação (180 CU, de 720 para 900). O manuseio dos artefatos maiores do Falcon-512 (assinatura de 666 *bytes* e chave pública de 897 *bytes*) acrescenta apenas cerca de 58 CU. Isso confirma que, equalizada a arquitetura, o sobrecusto do fluxo pós-quântico é pequeno e dominado pela própria primitiva, e não pelo tamanho dos dados pós-quânticos.

TABELA IV

DECOMPOSIÇÃO MEDIDA DO CONSUMO DE *Compute Units* POR COMPONENTE ($N = 10.000$), PARA O CENÁRIO Ed25519 SOB ARQUITETURA EQUIVALENTE E O FLUXO FALCON-512. OS COMPONENTES ARQUITETURAIS SÃO PRATICAMENTE IDÊNTICOS, E A DIFERENÇA CONCENTRA-SE NA PRIMITIVA DE VERIFICAÇÃO.

Componente	Ed25519 (mesma arq.)	Falcon-512
<i>Runtime</i> BPF + <i>entrypoint</i>	3.850	3.850
Parse da instrução	21	24
Leitura da PDA + derivação (<i>create_program_address</i>)	1.700	1.748
Hash da mensagem + <i>logs</i>	723	724
Verificação (primitiva via <i>syscall</i>)	720	900
Transferência + epílogo	648	654
Total	7.662	7.900

Comparativo de três cenários de transferência
Benchmark Solana, $n = 10.000$ transações

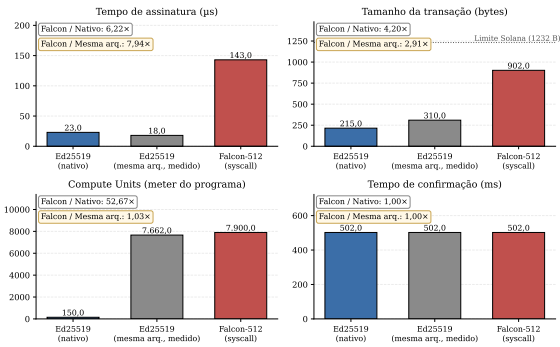


Fig. 1. Comparação entre Ed25519 nativo, Ed25519 sob mesma arquitetura (*syscall* dedicado, medido) e Falcon-512 via *syscall* para $N = 10.000$ transações. Cada painel apresenta o fator de *overhead* associado.

TABELA V

ESTATÍSTICAS DESCRITIVAS POR MÉTRICA E CENÁRIO ($N = 10.000$). DP = DESVIO PADRÃO, IC = INTERVALO DE CONFIANÇA DE 95% DA MÉDIA, MD = MEDIANA.

Cenário	Média ± DP	IC 95%	Md	P99
<i>Tempo de assinatura (μs)</i>				
Ed25519	23,1 ± 3,5	[23,1; 23,2]	23	33
Ed25519 (m.a.)	19,5 ± 18,8	[19,2; 19,9]	18	47
Falcon-512	150,3 ± 56,6	[149,2; 151,4]	143	298
<i>Compute Units</i>				
Ed25519	150 ± 0	[150; 150]	150	150
Ed25519 (m.a.)	7.662 ± 1,9	[7.662; 7.663]	7.663	7.663
Falcon-512	7.900 ± 1,9	[7.900; 7.901]	7.901	7.901
<i>Tamanho da transação (bytes)</i>				
Ed25519	215 ± 0	[215; 215]	215	215
Ed25519 (m.a.)	310 ± 0	[310; 310]	310	310
Falcon-512	902,1 ± 2,2	[902,0; 902,1]	902	907
<i>Tempo de confirmação (ms)</i>				
Ed25519	524,6 ± 104,0	[522,6; 526,6]	502	1.005
Ed25519 (m.a.)	528,5 ± 111,9	[526,3; 530,7]	502	1.005
Falcon-512	528,8 ± 111,9	[526,6; 531,0]	502	1.005

VI. DISCUSSÃO

Uma vez deslocada para o *syscall*, a verificação criptográfica passa a apresentar comportamento compatível com o ambiente avaliado. O custo isolado do Falcon-512 (900 CU) representa um acréscimo de apenas 180 CU (+25%) sobre o *SIGNATURE_COST* do Ed25519 (720 CU), preço direto da resistência quântica. O *overhead* total do fluxo, contudo, é muito maior (7.900 CU), porque a maior parte do custo não está na primitiva, mas em executar a verificação dentro de um programa BPF, o que envolve *runtime*, leitura e derivação da PDA, *logs* e lógica de autorização *on-chain*, custo esse praticamente idêntico ao do fluxo Ed25519 equivalente (7.662 CU). O principal gargalo da abordagem, portanto, não está na verificação Falcon-512 em si, mas na integração da camada de verificação ao modelo de execução da Solana.

A comparação sob arquitetura equivalente reforça essa interpretação. Quando o Ed25519 é executado sob o mesmo fluxo (PDA, *syscall* dedicado, transferência direta), o consumo **medido** é de 7.662 CU, indicando paridade técnica ($1,03\times$) com o fluxo Falcon-512 (7.900 CU). O diferencial percebido entre Ed25519 nativo (150 CU) e Falcon-512 (7.900 CU) tem, portanto, origem majoritariamente na transição do modelo de *sigverify* nativo para a verificação dentro de programa BPF (efeito de cerca de $51\times$), e não na escolha algorítmica entre Ed25519 e Falcon-512, cuja diferença sob arquitetura idêntica é de apenas 238 CU.

A equivalência entre tempos de confirmação deve ser interpretada com cautela, já que o *solana-test-validator* opera sem consenso distribuído nem propagação multi-validador. As métricas mais robustas são o tempo de assinatura, o tamanho das transações e os *Compute Units*, que dependem do algoritmo, do *payload* e do *runtime*, não do número de nós.

VII. LIMITAÇÕES E AMEAÇAS À VALIDADE

Os resultados apresentados devem ser interpretados em referência ao **ambiente single-node** utilizado. O *solana-test-validator* opera como nó único, sem consenso distribuído e sem propagação real via *Turbine* ou *Gulf Stream*. Por isso, métricas como tempo de confirmação

oferecem apenas uma aproximação parcial do comportamento de uma rede Solana em produção. Em particular, o aumento de $4,2\times$ no tamanho da transação (de 215 para 902 *bytes*) pode impactar a vazão em ambiente multi-validador, mas esse efeito não pôde ser medido no *solana-test-validator*. Há também uma questão de determinismo relevante para o consenso, pois a verificação Falcon-512 depende de aritmética de ponto flutuante de dupla precisão e exige resultado bit a bit idêntico entre validadores, condição que o ambiente *single-node* não exercita e que em *mainnet* demandaria uma verificação especificada de forma bit-exata.

Há ainda uma delimitação de escopo de segurança a explicitar. A garantia pós-quântica aplica-se aos ativos custodiados pelo programa, que são movidos a partir da PDA mediante verificação Falcon-512, e não ao saldo nativo de contas Ed25519 arbitrárias. Como o *payer* e o envelope da transação permanecem assinados por Ed25519, fundos fora do controle do programa continuam sujeitos ao modelo de segurança clássico, de modo que a coexistência protege explicitamente o que é colocado atrás da verificação pós-quântica.

Uma segunda limitação diz respeito à **governança e à adoção em *mainnet***. A inclusão do *syscall* no *runtime* oficial exige aprovação via SIMD (*Solana Improvement Document*), voto da maioria dos validadores e atualização de *bindings* em todo o ecossistema (SDKs, *wallets*, RPCs e *frameworks* como Anchor), coordenação que está fora do escopo técnico do estudo e representa uma barreira concreta entre a viabilidade demonstrada e a adoção efetiva.

Por fim, na **comparação sob arquitetura equivalente**, os valores absolutos de *Compute Units* dependem da versão do *runtime* e da *toolchain*, e os resultados não devem ser generalizados a outros algoritmos pós-quânticos, a outras parametrizações do Falcon (como Falcon-1024) ou a outras estratégias de integração nativa.

CONCLUSÃO

Este trabalho avaliou uma abordagem de coexistência criptográfica para a rede Solana, na qual o Ed25519 permanece como mecanismo principal e o Falcon-512 é implementado como uma camada adicional de autenticação. A principal contribuição foi mostrar que a verificação Falcon-512 diretamente na SVM encontra limitações relevantes, enquanto que sua execução via *syscall* nativo no *runtime* do validador se mostra tecnicamente viável no ambiente experimental. Além disso, os resultados revelaram que o custo adicional da abordagem concentra-se no *overhead* arquitetural de integração, e não na primitiva criptográfica em si.

A comparação sob arquitetura equivalente reforça essa conclusão, pois o consumo **medido** do Ed25519 sob o mesmo fluxo (7.662 CU) mantém paridade técnica ($1,03\times$) com o Falcon-512 (7.900 CU), com diferença de apenas 238 CU concentrada na primitiva de verificação. Esse resultado contradiz a percepção de que esquemas pós-quânticos seriam proibitivamente caros e localiza o desafio de adoção no plano arquitetural e de governança, não no plano algorítmico.

Conduzidos em ambiente *single-node*, os experimentos recomendam cautela quanto a propagação e consenso em rede

real. A contribuição não é propor uma migração completa da Solana, mas oferecer evidência experimental de que a coexistência híbrida com verificação nativa é tecnicamente plausível.

Como direções futuras, destacam-se a avaliação do mecanismo em ambiente multi-validador, capaz de caracterizar o impacto do aumento de *payload* sobre vazão e propagação, a comparação experimental direta com o precompilado SIMD-0461 [17] sob métricas equivalentes, e o estudo do processo de governança necessário para incluir o *syscall* no *runtime* oficial da Solana.

REFERENCES

- [1] P. W. Shor, “Polynomial-Time Algorithms for Prime Factorization and Discrete Logarithms on a Quantum Computer,” *SIAM Journal on Computing*, vol. 26, no. 5, pp. 1484–1509, 1997.
- [2] M. Mosca, “Cybersecurity in an Era with Quantum Computers: Will We Be Ready?” *IEEE Security & Privacy*, vol. 16, no. 5, pp. 38–41, 2018.
- [3] D. Aggarwal, G. K. Brennen, T. Lee, M. Santha, and M. Tomamichel, “Quantum attacks on Bitcoin, and how to protect against them,” *Ledger*, vol. 3, 2018.
- [4] D. J. Bernstein, N. Duif, T. Lange, P. Schwabe, and B. Y. Yang, “High-speed high-security signatures,” *Journal of Cryptographic Engineering*, vol. 2, no. 2, pp. 77–89, 2012.
- [5] A. Yakovenko, “Solana: A new architecture for a high performance blockchain v0.8.13,” 2018. Available: <https://solana.com/solana-whi-tepaper.pdf>
- [6] Solana Foundation, “Transactions: Instruction Format,” Solana Documentation. Available: <https://solana.com/pt/docs/core/transactions#instruction-format>
- [7] Solana Documentation, “Program Derived Addresses (PDAs).” Available: <https://solana.com/docs/core/pda>
- [8] National Institute of Standards and Technology, “Post-Quantum Cryptography: FIPS Approved,” 2024. Available: <https://csrc.nist.gov/news/2024/postquantum-cryptography-fips-approved>
- [9] Falcon Project, “Falcon: Fast-Fourier Lattice-based Compact Signatures over NTRU.” Available: <https://falcon-sign.info/>
- [10] Solana Documentation, “Limitations,” Solana Documentation. Available: <https://solana.com/docs/programs/limitations>
- [11] Z. Yang, H. Alfauri, B. Farkiani, R. Jain, R. Di Pietro, and A. Erbad, “A Survey and Comparison of Post-quantum and Quantum Blockchains,” *arXiv preprint arXiv:2409.01358*, 2024.
- [12] LACChain, “On-Chain Verification of Post-Quantum Signatures,” LAC-Net Documentation. Available: <https://lacnet.com/on-chain-verification-of-post-quantum-signatures>
- [13] M. Eum, J. Leal, and A. Velasco, “EIP-7619: Precompile Falcon512 Generic Verifier,” Ethereum Improvement Proposals, 2024. Available: <https://eips.ethereum.org/EIPS/eip-7619>
- [14] M. Eum, J. Leal, and A. Velasco, “EIP-8052: Precompile for Falcon Support,” Ethereum Improvement Proposals, 2025. Available: <https://eips.ethereum.org/EIPS/eip-8052>
- [15] Algorand Foundation, “Technical Brief: Quantum-Resistant Transactions on Algorand with Falcon Signatures,” 2025.
- [16] T. Pornin, “New Efficient, Constant-Time Implementations of Falcon,” Technical report, 2019. Available: <https://falcon-sign.info/falcon-impl-20190918.pdf>
- [17] M. Resnick and S. Kim, “Securing Solana Against a Powerful Quantum Adversary,” Anza, 2026. Available: <https://www.anza.xyz/blog/securing-solana-against-a-powerful-quantum-adversary>
- [18] D. Rubin, E. Cesena, and M. Latif, “Quantum Migration Paths for Solana,” Jump Crypto, 2026. Available: <https://jumpcrypto.com/resources/quantum-migration-paths-for-solana>